

Taller de Sistemas Embebidos – 2025 1C – Parcial

P1 Apellidos: Aguilar Espinoza Nombres: María Isabel
Aciertos (suman): 35 Yerros (restan): 8 Nota: 5/40

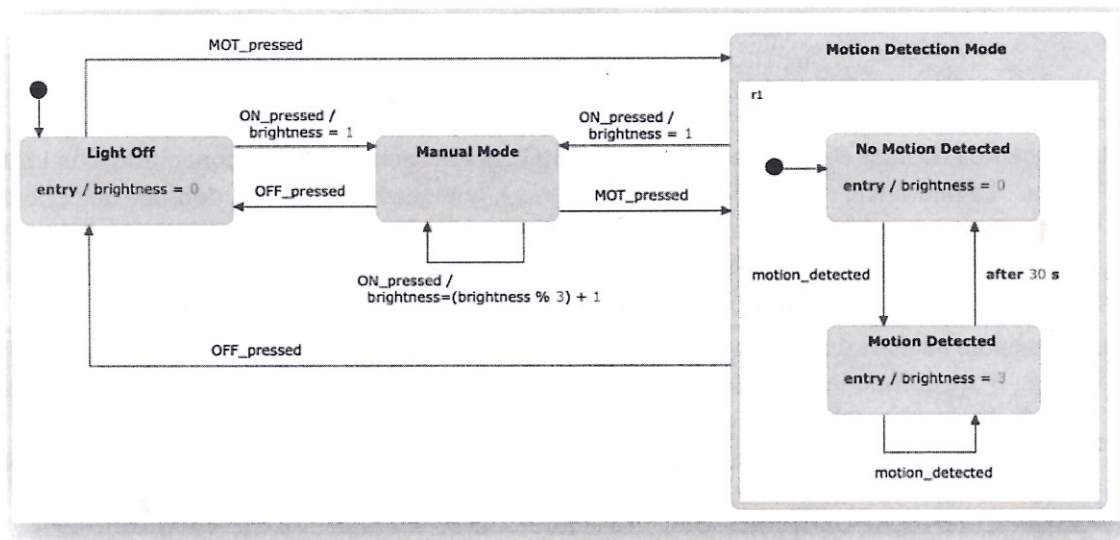
Favor de indicar en cada punto **una única respuesta**, la que considere **verdadera**.

1. Durante el Diseño/Desarrollo/Depuración del circuito eléctrico del **SE**, se recomienda dibujar:
 - ☒ a. Un diagrama de bloques de las funciones principales y un diagrama de c/función principal
 - b. Un único diagrama conteniendo todas las funciones principales (con diagrama de bloques)
 - c. Un único diagrama conteniendo todas las funciones principales (sin diagrama de bloques)
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
2. Los componentes básicos de una **computadora** son:
 - a. Unidad de Entrada
 - b. Unidad Central de Procesamiento (CPU)
 - ☒ c. Unidad de Salida
 - ☒ d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
3. Las características de la programación **Modular** son:
 - a. Código no estructurado
 - ☒ b. Subdividir un programa en subprogramas separados (módulo reutilizable)
 - c. Dificultad de uso (y reuso), mantenimiento de leer/entender/usar/developar/depurar/mantener
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
4. Como solución a Sistemas Embebidos (**SE**) orientados a **Control**, tenemos:
 - a. Disparador (trigger) es un suceso (evento/sincronismo) que inicia acción del Sistema de Control
 - b. La acción puede ser la ejecución de una tarea encargada de leer/calcular/enviar mensaje
 - ☒ c. Un evento significativo puede resultar de una "condición de programa/encuesta/interrupción"
 - ☒ d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
5. Para el Diseño/Desarrollo/Depuración de **Hardware** de **SE**, recurriremos a:
 - a. Herramientas Informáticas CAE: captura esquemática, simulación, depuración, validación y ...
 - b. Módulos Electrónicos: Placa de desarrollo: STM32 32-bits ARM Cortex MCUs, NUCLEO board
 - ☒ c. Dip-switchs, Buttons, Keyboards, Leds, LCD Displays, etc. e instrumentos de Laboratorio
 - ☒ d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
6. Para el Diseño/Desarrollo/Depuración de **Software** de **SE**, la Aplicación (escutar, procesar, actuar):
 - a. Los módulos se comunican / sincronizan mediante: Flags o Variables o Colas
 - b. Garantizar comportamiento comunitario (que ningún módulo se apropie de la CPU)
 - c. El código bloqueante es inaceptable
 - ☒ d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
7. ¿Qué es Itemis **CREATE**?
 - a. Es un sistema de gestión de código fuente y control de versiones distribuido
 - ☒ b. Es un servicio de alojamiento (hosting) para repositorios
 - ☒ c. Es un sistema de desarrollo, que usaremos para desarrollar/depurar Diagramas de Estado
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
8. En una arquitectura de computadora, el **bus de control** sirve para:
 - a. Derivar de la unidad solicitante (procesador), especificar a qué unidad/elemento acceder
 - b. Transferir información entre la unidad solicitante y otra unidad
 - ☒ c. Especificar qué hacer: dirección de las transferencias (lectura o escritura) y cuándo hacerlo
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)

+8 - 0

9. Los usuarios de aplicaciones quieren un producto terminado (**Microcontrolador**):
- Potente en términos de lo que quieren hacer y disponible en el mercado (circuito integrado)
 - Pueden elegir entre miles de dispositivos de cientos de fabricantes
 - Trabajan al nivel de la placa de circuito impreso (PCB)
 - Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
10. Cuando un ingeniero tiene que diseñar una nueva solución **SoC**, primero genera un prototipo:
- Que no permite la verificación de la cantidad de memoria necesaria para el proyecto
 - Que no permite la verificación del tiempo de respuesta del sistema
 - El producto final no dependerá de un SDK, pero un SDK ayudará en su diseño
 - Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
11. Para **guardar y ejecutar** las operaciones, se utilizan **registros** como **fuentes y destinos de datos**:
- En los procesadores modernos pueden haber 16, 32 o incluso más
 - Para escribir o leer registros, el acceso se realiza a través de uno o más buses de datos externos
 - Un registro particular como registro de flags o registro de estado
 - Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
12. El **pipelining** trae problemas al ejecutar una instrucción de **bifurcación** (Branch o Jump):
- La dirección de la siguiente instrucción se conocerá al completar la ejecución de la siguiente
 - No es necesario retrasar la siguiente recuperación ni interrumpir el pipeline
 - Es positivo porque un programa tiene muchas más instrucciones que bifurcaciones
 - Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
13. Para transferir instrucciones/datos dentro y fuera del **procesador**, se requiere de dos o más buses:
- von Newman**, instrucciones y datos comparten el mismo par de bus de direcciones y datos
 - Harvard**, instrucciones y datos no comparten el mismo par de bus de direcciones y datos
 - von Newman** es menos potente en términos de eficiencia informática que **Harvard**
 - Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
14. Cuando el procesador **ARM Cortex-M** comienza a funcionar (al encenderse o después de reiniciarse):
- Necesita ejecutar código para iniciar el sistema, denominado boot (strap) code
 - El vector de Reset apuntará a la dirección del programa de encendido o reinicio
 - La tabla de vectores de excepción comienza en la direcciones bajas de memoria no volátil
 - Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
15. Los tipos de **memorias** disponibles según su capacidad de retener o no datos son:
- ROM/PROM/EPROM/OTP EPROM **volátiles**
 - EEPROM/FLASH/FRAM/MRAM **volátiles**
 - SRAM/SSRAM **volátiles**
 - Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
16. **Cortex-M0** y **Cortex-M0+** ofrecen:
- El consumo de energía más alto de la familia **Cortex-M**
 - El rendimiento mayor de la familia **Cortex-M**
 - El conjunto de instrucciones más grande de la familia **Cortex-M**
 - Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
17. Los diagramas de estado de **Harel**, introducen un enfoque:
- Modular, jerárquico y bien estructurado
 - Con estados compuestos, con ortogonalidad (paralelismo)
 - Con comunicación (c/sincronización)
 - Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-

+8 -1



Extended light switch example as a Harel statechart with composite states

18. Para el diagrama de estado de Harel de la figura, al recibir la siguiente secuencia de entradas:

Indicar por qué **estados** evolucionará el diagrama luego de ser reiniciada:

- | | | | | | | |
|--------------------|----|--------------------|---|-----------------------------------|---|----------------|
| a. Reset | => | Light Off | / | entry | / | brightness = 0 |
| b. ON_pressed | => | Manual Mode | / | brightness = 1 | / | |
| c. ON_pressed | => | Manual Mode | / | brightness = (brightness % 3) + 1 | / | |
| d. ON_pressed | => | Manual Mode | / | brightness = (brightness % 3) + 1 | / | |
| e. MOT_pressed | => | No Motion Detected | / | entry | / | brightness = 0 |
| f. motion_detected | => | Motion Detected | / | entry | / | brightness = 3 |
| g. ON_pressed | => | No Motion Detected | / | after 30 s | / | |
| h. OF_pressed | => | Light Off | / | entry | / | brightness = 0 |
| i. fin | | | | | | |

19. Para el diagrama de estado de Harel de la figura, al recibir la siguiente secuencia de entradas:

Indicar por qué valores evolucionará la variable **briggthness** (estado a estado) luego de ser reiniciada:

- | | | |
|--------------------|----|---------|
| a. Reset | => | 0 |
| b. ON_pressed | => | 1 |
| c. ON_pressed | => | 1, 1 |
| d. ON_pressed | => | 1, 1, 1 |
| e. MOT_pressed | => | 0 |
| f. motion_detected | => | 3 |
| g. ON_pressed | => | 0 |
| h. OF_pressed | => | 0 |
| i. fin | | |

20. Conceptos básicos de **puertos de entrada/salida** digital del **Microcontrolador**:

- Los pines de un puerto de **entrada/salida** digital pueden cambiar de dirección
- Ocasionalmente el **Microcontrolador** envía datos a otro **IC** (opera como salida)
- Ocasionalmente el **Microcontrolador** recibe datos a otro **IC** (opera como entrada)
- Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)

21. Conceptos básicos de **puertos de salida** digital del **Microcontrolador**, conectado a una carga:

- El **uC** (fuente) drena la corriente para accionar el dispositivo (carga), push-pull u open drain
- La corriente fluye de la alimentación a través de la carga y del pin de salida del **uC** a tierra
- El límite de corriente (I_{OL}) dependerá del tipo de **uC** y del pin específico (validar c/hoja de datos)
- Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)

+2 -0

22. Conceptos básicos de **puertos de salida** digital del **Microcontrolador**, conectado a un **LED**:
- a. La diferencia de potencial en el **LED** (V_{FWD}), polarizado en directa, decrece c/la corriente (leve)
 - b. Una conexión válida es ánodo a alimentación (vía R_{SERIE}) y cátodo al pin de salida digital del **uC**
 - c. Una conexión válida es ánodo a tierra y cátodo al pin de salida digital del **uC** (vía R_{SERIE})
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
23. Conceptos básicos de **puertos de salida** digital del **Microcontrolador**, conectado a un **FET** canal N:
- a. Para componentes que requieren más corriente o tensión que los que pueden manejar el **uC**
 - b. El pin de salida digital del **uC** se conecta al pin gate del **FET** canal N
 - c. El pin drain del **FET** canal N se conecta a alimentación (vía carga) y el pin source a tierra
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
24. Conceptos básicos de **puertos de entrada/salida** digital del **Microcontrolador**, contacto húmedo:
- a. (con tensión) es un interruptor en que la tensión la suministra una fuente interna
 - b. Se conocen como contactos pasivos
 - c. Proporciona aislamiento y seguridad esenciales en los sistemas eléctricos
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
25. El diagrama de bloque de **STM32FXXXYY**, **HAL_GPIO** Module:
- a. El pin de **GPIO** no tiene conectados circuitos robustos de protección contra sobretensiones
 - b. El pin de **GPIO** no está protegido hasta un nivel de tensión de entrada aplicado de 5V
 - c. El pin de **GPIO** está conectado simultáneamente a los controladores de entrada y salida
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
26. El diagrama de bloque de **STM32FXXXYY**, **HAL_GPIO** Module, **controlador de salida** (opciones):
- a. El pin de **GPIO** tiene velocidad de operación: Low (válida)
 - b. El pin de **GPIO** tiene velocidad de operación: Medium (no válida)
 - c. El pin de **GPIO** tiene velocidad de operación: High (no válida)
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
27. El diagrama de bloque de **STM32FXXXYY**, **HAL_GPIO** Module, **controlador de entrada/salida**:
- a. La **HAL** no se basa en la estructura C **GPIO_InitStruct** para configurar cualquier pin de **GPIO**
 - b. El método **HAL_GPIO_TogglePin()** fuerza a bajo el nivel de un pin de **GPIO**
 - c. El método **HAL_GPIO_EXTI_Callback()** conectado al **IRQ EXTI** de un pin de **GPIO**
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
28. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **Interrupciones**:
- a. No puede ser de hardware, como es el caso cuando se trata de un temporizador
 - b. Puede ser de software (fin anormal de programa/dividir por cero/acceso a memoria inexistente)
 - c. En la terminología ARM, todas las interrupciones, de hardware o software, son excepciones
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
29. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **NVIC**:
- a. Conectado dentro del núcleo **Cortex-M**
 - b. Compatible con **CMCIS** (mismo tipo de registros de control/configuración que **GPIO**)
 - c. No integrado al software **HAL** y usa sus propias estructuras C para configurar sus registros
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
30. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **NVIC**:
- a. Armar: permite que un dispositivo de hardware active una interrupción
 - b. Habilitar: Permite interrumpir el proceso
 - c. Deshabilitar: No permite el proceso de interrupción
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
31. El diagrama de bloque de **STM32FXXXYY**, **NVIC**, **eventos posteriores** al reconocimiento de una **IRQ**:
- a. La **NVIC** interactúa automáticamente con la **MCU** usando firmware almacenado dentro de ella
 - b. Sólo se necesitan 18 ciclos de reloj, para pasar de reconocer una **IRQ** a ejecutar la **ISR**
 - c. Tiempo de latencia de aproximadamente **150µs** para una **MCU** con frecuencia de **80MHz**
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)

32. El diagrama de bloque de **STM32FXXXYY**, **NVIC**, la **ISR**:
- a. Hay una lista de direcciones para estas pequeñas **ISR** almacenadas en una tabla de vectores
 - b. Esta característica da lugar a la palabra "vector" en el nombre de **NVIC**
 - c. **STM** usa encuesta (polling) para pines GPIO del mismo puerto que comparten el mismo vector
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
33. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **Timers**:
- a. Tienen muchos usos, generar señales precisas de modulación de ancho de pulso (**PWM**)
 - b. Tienen muchos usos, generar pulso único con longitud programable y características de retardo
 - c. Tienen muchos usos, generar señales periódicas de acceso directo a memoria (**DMA**)
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
34. El diagrama de bloque de **STM32FXXXYY**, **Timers**, la categoría de timers **STM** de **Alta resolución**:
- a. Múltiples salidas de alta resolución posibles (seis subtimers)
 - b. Con este temporizador son posibles múltiples inserciones de tiempo muerto
 - c. Este temporizador tiene el acrónimo **HRTIM1** en terminología **STM**
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
35. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **ADC**:
- a. La secuencia se repite hasta que se alcanza y procesa el bit menos significativo (**LSB**)
 - b. Luego de lo cual, el **N-BIT REGISTER** contendrá el equivalente digital completo de **V-IN**
 - c. El módulo **SAR LOGIC** permitirá que el valor digital convertido se libere en paralelo o serie
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
36. El diagrama de bloque de **STM32FXXXYY**, **ADC**, **Single Channel/Continuous Conversion**
- a. Es el modo más simple
 - b. Se muestrean múltiples líneas analógicas de entrada y luego se convierte en números digitales
 - c. Los números digitales resultantes se almacenan contínuos en la memoria
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
37. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **ADC**:
- a. El método **HAL_ADC_Stop()** detiene la conversión
 - b. El método **HAL_ADC_GetValue()** lee la conversión
 - c. El método **HAL_ADC_ConvCpltCallback()** conectado al **IRQ** de un **ADC**
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
38. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **Timer PWM**:
- a. La **HAL** se basa en la estructura **C TIM_OC_InitTypeDef** para configurar cualquier **Timer PWM**
 - b. El método **HAL_TIM_IRQHandler()** maneja un **IRQ** de un **Timer**
 - c. El método **HAL_TIM_PWM_PulseFinishedCallback()** conectado al **IRQ** de un **Timer**
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
39. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **SPI**:
- a. Es de las interfaces de comunicación del tipo "Intra System Protocol"
 - b. Es de las interfaces de comunicación del tipo "asíncronica"
 - c. Es de las interfaces de comunicación del tipo "half duplex", "bidireccional"
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
40. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **SPI**:
- a. Datos transmitidos entre el nodo principal y el subnodo se sincronizan con el reloj del subnodo
 - b. Los dispositivos **SPI** admiten frecuencias de reloj mucho más altas que las interfaces **I2C**
 - c. La conexión de líneas entre el nodo principal y el subnodo es: **/CS => SCLK**
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
41. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **I2C**:
- a. Es de las interfaces de comunicación del tipo "Inter System Protocol"
 - b. Es de las interfaces de comunicación del tipo "asíncronica"
 - c. Es más lenta que la interfaz **SPI**
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-

44 - 4

42. El diagrama de bloque de **STM32FXXXYY**, usado en la placa **NUCLEO-FXXXZZ (SDK)**, **I2C**:
- a. El protocolo sólo permite conectar dispositivos de igual velocidad al **Microcontrolador**
 - b. Las líneas **SDA** y **SCL** son del tipo "open-colector/open-drain"
 - c. La señal de **reloj** siempre está controlada por el subnodo
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
43. TP0-05 - 3er Proyecto p/placa **NUCLEO-F103RB** (Event Driven System (**EDS**) -...), en la práctica: Un ejecutor cíclico de tareas, con estructuras de configuración (**cfg**) y datos (**dta**), se observó la evolución del campo **WCET**, Worst-Case Execution Time
- a. **task_a**, **Blocking Code**, se ejecuta de forma asincrónica (no se ejecuta línea por línea)
 - b. **task_b**, **Non-Blocking Code**, se ejecuta de forma sincrónica (se ejecuta línea por línea)
 - c. **task_c**, **Update by Time Code**, se ejecuta por ocurrencia de ticks
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
44. TP0-05 - 3er Proyecto p/placa **NUCLEO-F103RB** (Event Driven System (**EDS**) -...), en la práctica:
- a. **WCET**, Worst-Case Execution Time, se mide en nano Segundos
 - b. **WCET**, Worst-Case Execution Time, se mide en mili Segundos
 - c. **WCET**, Worst-Case Execution Time, se mide en Segundos
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
45. TP0-05 - 3er Proyecto p/placa **NUCLEO-F103RB** (Event Driven System (**EDS**) -...), en la práctica:
- a. El método **HAL_SYSTICK_Callback**, inicializa **g_task_test_tick_cnt**
 - b. El método **HAL_SYSTICK_Callback**, decrementa **g_task_test_tick_cnt**
 - c. El método **HAL_SYSTICK_Callback**, incrementa **g_task_test_tick_cnt**
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
46. TP1-01 - **My First Statechart**, en la práctica: Al ingresar por primera vez a **Itemis Create Cloud Editor**
=> se genera automáticamente el modelo **My First Statechart**
- a. Un **statechart** puede tener variables, pero no modificarlas
 - b. Un **evento** puede tener una guarda que opera como una proposición "OR"
 - c. La **sincronización** entre **statecharts** es mediante el "raise" de un **evento**
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
47. TP1-01 - **My First Statechart**, en la práctica: Al ingresar por primera vez a **Itemis Create Cloud Editor**
- a. Un **statechart** no necesita tener **pseudo estado** inicial
 - b. Un **statechart** no necesita tener **pseudo estado** final
 - c. Una **transición** puede tener un **pseudo estado** choise que opera como una proposición "XOR"
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
48. TP1-01 - **My First Statechart**, en la práctica: Al ingresar por primera vez a **Itemis Create Cloud Editor**
- a. Un **statechart** debe tener **pseudo estado** final
 - b. Un **statechart** no necesita tener **pseudo estado** inicial
 - c. Un **estado compuesto** debe tener **pseudo estado** inicial
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
49. TP2-01 - 4to Proyecto p/placa **NUCLEO-F103RB** (**Sensor Statechart**) (1 sensor), en la práctica: Al codificar la tarea modelo del sensor: **Sensor Statechart**
- a. Cada sensor produce dos **eventos** de la máquina de estado
 - b. Cada estado produce un "case" en el "switch" de la máquina de estado
 - c. Cada evento produce un "if" en el "case" del estado al que afecta/excita
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)
-
50. TP2-01 - 4to Proyecto p/placa **NUCLEO-F103RB** (**Sensor Statechart**) (**N** sensor), en la práctica: Al codificar la tarea modelo del sensor: **Sensor Statechart**
- a. Los arrays de **estructuras** de configuración (**cfg**) y datos (**dta**), son de **N-1** elementos c/u
 - b. Los arrays de **estructuras** de configuración (**cfg**) y datos (**dta**), son de **N-2** elementos c/u
 - c. Los arrays de **estructuras** de configuración (**cfg**) y datos (**dta**), son de **N-3** elementos c/u
 - d. Ninguna de las anteriores o Todas las anteriores (lo que aplique al caso)